

Chapter 2

Data:

Data is a collection of raw facts, figures, or information that may not have meaning on its own. It can be in the form of numbers, words, images, or symbols.

For example: your name, age, mobile number, or marks in exams are all types of data.

Information:

Information is the useful result we get after arranging or processing data. It helps us understand things better and make decisions.

Example:

If “80, 85, 90” are marks (data), then

“The student scored an average of 85 marks” is information.

Database:

A database is an organized collection of data that can be easily accessed, managed, and updated. It stores information in tables, making it easier to find and use. For example, in a school database, student details like names, grades, and attendance are stored. Databases help businesses, schools, and websites keep large amounts of data safe and easy to retrieve when needed.

DBMS:

A Database Management System (DBMS) is an application software designed to store, manage, and organize data efficiently. It allows us to create, retrieve, update, and delete data in a structured way, ensuring data integrity, security, and ease of access. A DBMS acts as an intermediary between users and the database, making data management seamless and secure.

E.g: MySQL, Oracle Database, Microsoft SQL Server, PostgreSQL, MongoDB, SQLite

Features of DBMS

Data Redundancy, Data Consistency, Data Backup and recovery, Data Integration, Data Security, Data Manipulation, Data retrieval, Data Storage

Benefits of DBM

1. Easy data storage and retrieval
2. Reduces data redundancy
3. Ensures data consistency and accuracy
4. Allows multiple user access

5. Supports data backup and recovery

Challenges of DBMS

1. Requires technical knowledge to manage
2. Initial setup cost can be high.
3. Data security risks if not properly managed.
4. Complex systems may need regular maintenance.
5. Performance may be slow with very large data.

Relational Database Management System (RDBMS):

Relational Database Management System is a type of DBMS that stores data in tables made up of rows and columns. Each table is linked to others through common fields, which makes it easy to organize and manage related information.

Data Types

In SQL, data types define the kind of data that can be stored in each column of a database table.

1. Numeric Data Types

Numeric Data Types are used for storing numbers in a database.

1.1. INT (Integer)

- Used to store whole numbers without decimal points.

Storage Size:

- 4 Bytes (32 bits)

Range:

- -2,147,483,648 to 2,147,483,647

Example Values:

10, 25, 100

1.2.SMALLINT

- Used to store small-range whole numbers.

Storage Size:

- 2 Bytes (16 bits)

Range:

- -32,768 to 32,767

Example Values:

5, 200, 1000

1.3.BIGINT

- Used to store very large whole numbers.

Storage Size:

- **8 Bytes (64 bits)**

Range:

- Very large values:
-9,223,372,036,854,775,808
to
9,223,372,036,854,775,807

Example Values:

9876543210

1.4. DECIMAL (p,s)

- Used to store exact decimal numbers.

Storage Size:

- Depends on precision (p) and scale (s)

Meaning:

- p = total number of digits
- s = digits after decimal point

Example:

DECIMAL(5,2)

Can store:

123.45

Cannot store:

12345.67

1.5. FLOAT

- Used to store approximate decimal numbers.

Storage Size:

- **4 Bytes**

Example Values:

3.14
0.00045

2. Character (Text) Data Types

Character Data Types are used for storing text values.

2.1.CHAR (n)

- Stores fixed-length text.

Storage Size:

- Exactly n characters.

Example:

CHAR (5)

If stored value is:
RAM

It becomes:
RAM

2.2.VARCHAR(n)

- Stores variable-length text.

Storage Size:

- Stores only actual characters used.

Example:

VARCHAR(20)

2.3.TEXT

- Used for storing long paragraphs or descriptions.

Storage Size:

- Can store very large text data.

3. Date and Time Data Types

Date and Time Data Types are used for storing dates and times.

3.1.DATE

- Stores only date.

Format:

YYYY-MM-DD

Storage Size:

- **3 Bytes**

Example:

2082-05-12

3.2.TIME

- Stores only time.

Format:

HH:MM:SS

Storage Size:

- **3 Bytes**

Example:

14:30:00

Common Uses:

- Class timing
- Meeting schedule

3.3.DATETIME

- Stores both date and time.

Storage Size:

- **8 Bytes**

Example:

2082-05-12 14:30:00

Common Uses:

- Login records
- Event details

3.4.TIMESTAMP

- Similar to DATETIME.
- Automatically records time-related events.

Storage Size:

- **4 Bytes**

Common Uses:

- Last updated time
- Login history

Important Points:

- Automatically updates in many systems.

4. Boolean Data Type**4.1.BOOLEAN**

- Stores only:
 - TRUE
 - FALSE

Storage Size:

- Usually **1 Byte**

Extra Important DBMS Data Types (Good to Know)

These are used in modern DBMS systems:

1. BINARY / VARBINARY

- Stores binary data (images, files)

Use:

- Images
- Documents
- Videos (rare in DB, usually stored externally)

2. ENUM (MySQL-specific)

- Stores predefined values only

Example:

ENUM('Male','Female','Other')

Use:

- Gender selection
- Status fields

3. JSON (Modern DBMS like MySQL, PostgreSQL)

- Stores structured data in JSON format

Example:

{"name":"Aarav","age":15}

Use:

- Web apps
- APIs
- Flexible data storage

Table

A **table** is the main structure used in a database to store data in the form of **rows and columns**.

It is also called the **foundation (building block) of a database**.

Key Features of Table

1. Rows and Columns

- Data is stored in horizontal **rows** and vertical **columns**.

2. Unique Rows (Records)

- Each row represents one complete record.
- Each row should be unique.

3. Schema (Structure)

- Each column has a fixed data type (INT, VARCHAR, DATE, etc.).

4. Organized Format

- Data is stored in a clean and structured way.

5. Easy Operations

- Data can be easily:
 - Added
 - Updated
 - Deleted
 - Searched

6. Relationships

- Tables can be connected using keys (Primary & Foreign keys).

Structure of a Table

A table contains:

- **Columns (Fields)**
- **Rows (Records)**

Example

Table: Student

Student_ID	Name	Class	Marks
101	Aarav	10	85
102	Sita	9	90

Important Terms in Table

1. Field (Column)

A **field** is a column in a table.

It stores a single type of information for all records.

Example:

Student_ID Name Class DOB

Here:

- Student_ID, Name, Class, DOB = Fields

Important Points of Field

- Field = Vertical column
- Each field stores one type of data
- Field name describes the data
- Each field has a data type

2. Record (Row)

A **record** is a row in a table.

It represents one complete set of data about an entity.

Example:

| 101 | Aarosh | 10 | 2010-03-12 |

This is ONE record.

Important Points of Record

- Record = Horizontal row
- Contains full information about one entity
- Made up of multiple fields
- Each row must be unique

Keys in Database

Keys are used to:

- Identify records uniquely
- Connect tables together

1. Primary Key

A **Primary Key** uniquely identifies each record in a table.

Important Features:

- Must be unique
- Cannot be NULL
- Only one primary key per table
- No duplicate values allowed

Example:

StudentID	Name	Class
101	Ram	10
102	Sita	9

☞ StudentID = Primary Key

2. Foreign Key

A **Foreign Key** is a field in one table that refers to the **Primary Key of another table**.

It is used to connect two tables.

Important Features:

- Creates relationship between tables
- Refers to Primary Key
- Maintains data integrity
- Can repeat values

Example:

Student Table

StudentID	Name
201	Nidhi
202	Sampada

Library Table

BookID	BookTitle	StudentID
B01	DBMS	201
B02	Python	202

StudentID in Library = Foreign Key

3. Composite Key

A **Composite Key** is formed by combining two or more fields to uniquely identify a record.

Example:

StudentID	Subject	Marks
301	Math	90
301	Science	85

Neither StudentID nor Subject alone is unique

But StudentID + Subject = Unique

Super Important Exam Difference

Key Type	Purpose
Primary Key	Uniquely identifies rows
Foreign Key	Links tables
Composite Key	Combines columns for uniqueness

Types of Relationships

Relationships show how tables are connected. They allow data to be stored efficiently, reduce redundancy, improve data integrity, and enable accurate retrieval of information.

There are three types of relationship in database they are:

1. One-to-One (1:1)

- In a one-to-one relationship, each record in one table is connected to only one record in another table.
- One record in Table A = One record in Table B

Example:

A person can have only one passport, and each passport is issued to only one person.

Person $\leftrightarrow$ Passport

2. One-to-Many (1:M)

- In a one-to-many relationship, a single record in one table is linked to multiple records in another table.
- One record in Table A $\rightarrow$ Many records in Table B

Example:

A teacher may teach many classes, but each class is taught by only one teacher.

Teacher $\rightarrow$ Students

3. Many-to-Many (M:M)

- In a many-to-many relationship, multiple records in one table can be related to multiple records in another table.
- Many records in both tables are related

Example:

Students $\leftrightarrow$ Courses

Needs a **junction table**

Real-Life Example Summary

Relationship	Example
1:1	Person–Passport
1:M	Teacher–Students
M:M	Students–Courses

Introduction to MySQL:

MySQL is a popular relational database management system (RDBMS) that uses SQL (Structured Query Language) to store, manage, and organize data. Originally developed by a Swedish company called MySQL AB, it is now maintained by Oracle Corporation.

Features of MySQL

- i. Simple language
- ii. Data retrieval
- iii. Data manipulation
- iv. Data definition
- v. Multiple users
- vi. Security features
- vii. Used in modern applications

Download MySQL

Step 1: Visit the Official MySql Download Page: <https://www.mysql.com/downloads/>

Step 2: Click on MySQL Community (GPL) Downloads

Step 3: Click on MySQL installer for windows from MySQL

Step 4: After performing the above operation on MySQL Installer for windows from MySQL, click on the second option Windows (x86, 32-bit), MSI Installer. [Note : you can change version and operating system as per your requirement]

Step 5: Now, Click on No thanks, just start my download.

Now, downloading process will proceed. After you download MySQL on your PC, you need to install it into your PC.

Step 6: Locate the installer file you just downloaded (e.g., mysql-installer-web- community-x.x.xx.x.msi) and double-click it to run the installer.

Step 7: In the installer window, choose “Full” under “Setup Type.” This option will install MySQL Server, MySQL Workbench and other components, such as documentation and examples. Then click “Next.”

Step 8: In the next window, just click “Execute.” After all products have been installed successfully, click “Next” to start the configuration.

Step 9: Under “Type and Networking”, choose “Development Computer” as Config Type and leave everything else as is. Then, click “Next.”

Step 10: Authentication method page appears, simply click on Next.

Step 11: In the “Account and Roles” window, choose a new password for your root account. Be sure to choose a strong password. Remember this password is used when you want to open MySQL using root login.

Step 12: After entering password, MySQL shows Windows Service leave everything as is and click Next.

Step 13: Next, grant full access to let the Installer update server file permissions and click “Next.” Click “Execute” to apply the configuration changes.

Before finishing, MySQL may ask you to test your connection. Click “Test Connection” to verify that MySQL Server is running properly.

Step 14: Once the configuration is done (green checkmarks), click “Finish.”

Using MySQL Workbench

At the end of the installation process, MySQL Workbench should launch automatically. If it doesn’t, start the program manually. You are greeted by the graphical user interface (GUI) tool for MySQL, the MySQL Workbench.

In the bottom half of the window under “MySQL Connections” you should see one tile for your local instance connection, Local instance MySQL80.

- a. Go ahead and double-click it.
- b. In the new window, enter the password you chose earlier and click “OK.”
- c. You’re now connected to your MySQL Server and ready to start using MySQL Workbench to work with your databases.

Verify MySQL Installation

To do this, open the command prompt (or terminal) and type:

```
mysql - - version
```

If MySQL is installed properly, this command will display the installed version of MySQL. For example: mysql Ver 8.0.34 for Win64 on x86_64

```
mysql -u root -p
```

This will prompt you to enter the root password. If you successfully log in and see the `mysql >` prompt, it means your MySQL server is running properly. This verification step ensures you are ready to start creating databases and tables.

Table in MySQL

In MySQL, a table is a basic structure used to store data in a database. All the data in a database is stored in one or more tables. You can create, modify, or delete tables using SQL commands.

Features of a table:

1. Stores data in rows and columns format.
2. Each column has a specific data type (e.g., number, text, date).
3. Each row in a table is called a record.
4. Each column in a table is called a field.
5. Tables can be linked with each other using keys (like Primary Key and Foreign Key).
6. Tables can be created, modified, or deleted using SQL queries.

Relational Table

A relational table is a table in a relational database where data is organized into rows and columns, and each table represents one type of entity (like students, teachers, or courses).

For an example, you might have:

- i. A 'students' table with student details.
- ii. A 'courses' table with course details.
- iii. A 'student_courses' table that relates students to the courses they are enrolled in.

Example of Relational Tables

Students Table

StudentID	Name
101	Aarav

Courses Table

CourseID	Course
C01	DBMS

Student_Courses Table

StudentID	CourseID
101	C01

This creates relationships between students and courses.

Structured Query Language (SQL)

Query is a request made to a database to retrieve, insert, update, or delete data. Queries are written using SQL (Structured Query Language), which was developed by IBM in the 1970s to manage data in relational databases. For example, if you want to find all students in Class 10 from your database, you can write a query to display only those records.

SQL (Structured Query Language) pronounced as “**See**”-“**quell**” is made of three sub languages DDL, DML and DCL.

SQL is used to:

- Create databases
- Create tables
- Insert data
- Update data
- Delete data
- Retrieve data

Query

A query is a request made to a database to perform operations like Retrieve data, Insert data, Update data and Delete data. Queries are written using SQL commands. Queries help users interact with databases efficiently.

Features of a Query

- i. Retrieves specific data from tables.
- ii. Adds, updates, or deletes records
- iii. Filters and sorts data easily
- iv. Useful for reports and analysis

Example of Query

If we want to display all students from Class 10:

```
SELECT * FROM students  
WHERE Class = 10;
```

This query retrieves only students who belong to Class 10.

SQL Sub Languages

SQL is divided into three main sub-languages:

Sub-language	Full Form
DDL	Data Definition Language
DML	Data Manipulation Language
DCL	Data Control Language

MySQL Constraints

Constraints are rules applied to table columns to control the type of data that can be entered into a table. When constraints are used properly, they prevent errors like duplicate values, missing important data, or incorrect entries.

Constraints help:

- Maintain data integrity
- Prevent invalid data
- Ensure accuracy and consistency

Types of MySQL Constraints

1. PRIMARY KEY

A PRIMARY KEY uniquely identifies each record in a table, Ensures no duplicate or NULL values are allowed in the specified column.

2. FOREIGN KEY

A FOREIGN KEY is used to create relationships between two tables. It refers to the primary key of another table.

3. NOT NULL

The NOT NULL constraint prevents a column from storing NULL or empty values.

Example:

A student name should never be empty.

4. UNIQUE

The UNIQUE constraint ensures all values in a column are different. Duplicate values are not allowed.

5. CHECK

The CHECK constraint limits the values that can be inserted into a column based on a condition.

Example:

Age must be greater than 0.

6. DEFAULT

The DEFAULT constraint automatically inserts a predefined value if no value is provided during insertion.

Importance of Constraints

1. Maintains data accuracy and consistency
2. Prevents duplicate and incorrect entries
3. Establishes relationships between tables
4. Enforces business rules

Data Definition Language (DDL)

DDL (Data Definition Language) is a part of SQL used to define and manage the structure of database objects such as:

- Tables
- Databases
- Views
- Indexes

DDL focuses on database structure rather than data itself.

Important Features of DDL

1. Defines database structure

DDL creates and manages database objects.

2. Automatically commits changes

DDL commands automatically save changes permanently.

3. Cannot usually be rolled back

Once executed, changes are difficult to undo.

DDL Commands

Command	Function
CREATE	Creates objects
ALTER	Modifies objects
DROP	Deletes objects
TRUNCATE	Removes all records
RENAME	Renames objects

1. CREATE Query

The CREATE query is used to create new database objects such as:

- Databases
- Tables

CREATE DATABASE Statement

Used to create a new database.

Syntax

```
CREATE DATABASE database_name;
```

Example

```
CREATE DATABASE school;
```

This query creates a database named school.

USE Command

The USE command selects a database for use.

Syntax

```
USE database_name;
```

Example

```
USE school;
```

This selects the school database.

CREATE TABLE Statement

Used to create a new table inside a database.

Syntax

```
CREATE TABLE table_name(  
    column1 datatype,  
    column2 datatype  
);
```

Example

```
CREATE TABLE students(  
    ID INT,  
    Name VARCHAR(50),  
    Class INT,  
    DOB DATE  
);
```

Explanation of Above Query

This query creates a table named students with four fields:

- ID → stores numbers
- Name → stores text
- Class → stores numbers
- DOB → stores date

Adding PRIMARY KEY while Creating Table

```
CREATE TABLE students(  
  ID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Class INT,  
  DOB DATE  
);
```

Explanation

- ID is set as PRIMARY KEY.
- Each ID must be unique.
- NULL values are not allowed.

Adding NOT NULL while Creating Table

```
CREATE TABLE students(  
  ID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Class INT NOT NULL,  
  DOB DATE  
);
```

Explanation

- Class field cannot remain empty.
- Every inserted record must contain a class value.

ALTER Query

The ALTER query is used to modify the structure of an existing table.

It can:

- Add columns

- Delete columns
- Modify columns
- Rename columns
- Remove constraints

1. Add New Column

Syntax

```
ALTER TABLE table_name  
ADD column_name data_type;
```

Example

```
ALTER TABLE students  
ADD Address VARCHAR(100);
```

This adds a new column named Address.

2. Drop/Delete Column

Syntax

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Example

```
ALTER TABLE students  
DROP COLUMN Address;
```

This deletes the Address column.

3. Modify Column

Syntax

```
ALTER TABLE table_name  
MODIFY COLUMN column_name new_data_type;
```

Example

```
ALTER TABLE students  
MODIFY COLUMN Name VARCHAR(100);
```

This changes the size of Name field.

4. Rename Column

Syntax

```
ALTER TABLE table_name  
CHANGE old_column_name new_column_name data_type;
```

Example

```
ALTER TABLE students  
CHANGE DOB DateOfBirth DATE;
```

This renames DOB to DateOfBirth.

DROP Query

The DROP query permanently deletes database objects.

Deleted objects cannot be recovered easily.

DROP DATABASE

Syntax

```
DROP DATABASE database_name;
```

Example

```
DROP DATABASE school;
```

This deletes the school database permanently.

DROP TABLE

Syntax

```
DROP TABLE table_name;
```

Example

```
DROP TABLE students;
```

This deletes the students table permanently.

Data Manipulation Language (DML)

DML stands for Data Manipulation Language.

It is used to work with data stored inside tables.

DML allows users to:

- Insert records
- Retrieve records
- Update records
- Delete records

Unlike DDL, DML focuses on data rather than table structure.

DML Commands

Command	Function
INSERT	Adds new records into a table
SELECT	Retrieves data from a table
UPDATE	Modifies existing records
DELETE	Removes records from a table

INSERT Query

The INSERT query is used to add new records (rows) into a database table.

It allows users to insert:

- Single record
- Multiple records

Whenever a new row is added into a table, the INSERT query is used.

Syntax of INSERT Query

```
INSERT INTO table_name(column1, column2, column3)
VALUES(value1, value2, value3);
```

Another Syntax

```
INSERT INTO table_name
VALUES(value1, value2, value3);
```

Example of INSERT Query

```
INSERT INTO students(ID, Name, Class, DOB)
VALUES(1, 'Dikshya Pokhrel', 10, '2007-05-10');
```

Explanation

This query inserts a new record into the students table.

Inserted values are:

- ID = 1
- Name = Dikshya Pokhrel
- Class = 10
- DOB = 2007-05-10

Example: Inserting Partial Values

```
INSERT INTO students(ID, Name)
VALUES(2, 'Binay Adhikari');
```

Explanation

This query inserts only:

- ID
- Name

Other columns remain NULL if allowed.

Important Points about INSERT Query

- 1. Used to add new records**
- 2. Values must match column data types**
- 3. Column order and value order must match**
- 4. String values are written inside single quotation marks (' ')**

SELECT Query

The SELECT query is used to retrieve or display data from one or more tables.

It is one of the most commonly used SQL queries.

The result returned by SELECT query is called a:

- Result Set

Syntax of SELECT Query

```
SELECT column_name  
FROM table_name;
```

Selecting All Columns

```
SELECT * FROM students;
```

Explanation

- * means ALL columns.
- This query displays every record and every column from the students table.

Selecting Specific Columns

```
SELECT Name FROM students;
```

Explanation

This query displays only the Name column from the students table.

Selecting Multiple Columns

```
SELECT Name, DOB  
FROM students;
```

Explanation

This query displays:

- Name
- DOB

from the students table.

Important Points about SELECT Query

1. Used to retrieve data
2. Can display all columns or selected columns
3. Can be used with WHERE clause
4. Does not modify data

UPDATE Query

The UPDATE query is used to modify existing records in a database table.

It helps:

- Correct mistakes
- Change values
- Update information

Syntax of UPDATE Query

```
UPDATE table_name  
SET column1 = value1, column2 = value2  
WHERE condition;
```

Example 1

```
UPDATE students  
SET Class = 11  
WHERE ID = 5;
```

Explanation

This query changes the Class of the student whose ID is 5 to 11. Only the matching row is updated.

Example 2

```
UPDATE student_tbl  
SET grade = 'B'  
WHERE name = 'Jaishree';
```

Explanation

This query updates the grade of Jaishree to B.

Important Points about UPDATE Query

1. Used to modify existing records
2. WHERE clause is important
3. Multiple columns can be updated together

Example:

```
UPDATE students  
SET Class = 10, Name = 'Ram'  
WHERE ID = 1;
```

DELETE Query

The DELETE query is used to remove records from a table.

It can:

- Delete selected rows
- Delete all rows

Syntax of DELETE Query

```
DELETE FROM table_name  
WHERE condition;
```

Example 1

```
DELETE FROM students  
WHERE Name = 'Muna';
```

Explanation

This query deletes the record where Name is Muna.

Example 2

```
DELETE FROM students  
WHERE ID > 5;
```

Explanation

This query deletes all records where ID is greater than 5.

Delete All Records

```
DELETE FROM students;
```

Explanation

This query removes all rows from the students table.

However:

- Table structure remains
- Columns remain
- Constraints remain

Important Points about DELETE Query

1. Used to remove records
2. WHERE clause prevents accidental deletion
3. Table structure remains safe

WHERE and LIKE Clauses

Using the SELECT query, we can retrieve or extract data from the database, based on specific criteria by using the WHERE and LIKE clauses.

WHERE Clause

The WHERE clause is used to filter records based on specific conditions.

It helps display or modify only required rows.

Without WHERE clause:

- All rows are affected.

Syntax of WHERE Clause

```
SELECT column_name  
FROM table_name  
WHERE condition;
```

Example using WHERE

```
SELECT Name, DOB  
FROM students  
WHERE Class = 10;
```

Explanation

This query displays:

- Name
- DOB

of students who are in Class 10 only.

WHERE Clause with UPDATE

```
UPDATE students  
SET Class = 11  
WHERE ID = 5;
```

Explanation

Only the student whose ID is 5 is updated.

WHERE Clause with DELETE

```
DELETE FROM students  
WHERE Name = 'Biraj';
```

Explanation

Deletes only the record whose Name is Biraj.

Important Points about WHERE Clause

1. Filters records based on conditions
2. Used with SELECT, UPDATE, and DELETE
3. Prevents unwanted changes

LIKE Clause

The LIKE clause is used to search for patterns in text values.

It is commonly used with:

- WHERE clause

LIKE helps search:

- Partial matches
- Similar text patterns

Wildcard Characters in MySQL

Wildcard	Meaning
%	Represents zero, one, or many characters
_	Represents exactly one character

Percent (%) Wildcard

% represents:

- Any number of characters

Example

```
SELECT Name, Class  
FROM students  
WHERE Name LIKE 'A%';
```

Explanation

This query displays students whose names start with letter A.

Examples:

- Aarav
- Aman
- Aayush

Underscore (_) Wildcard

_ represents:

- One unknown character

Example

```
SELECT Name  
FROM students  
WHERE Name LIKE '_man';
```

Explanation

This query displays names with:

- Exactly four letters
- Ending with "man"

Examples:

- Aman
- Iman

LIKE Clause with DELETE

```
DELETE FROM students  
WHERE Name LIKE '%esh';
```

Explanation

This query deletes names ending with:

- esh

Examples:

- Rajesh
- Ramesh

Important Points about LIKE Clause

1. Used for pattern searching
2. Works with text values
3. Uses wildcard characters

- %

- _

Difference between % and _ Wildcards

Wildcard	Meaning
%	Multiple characters
_	Single character

a) SELECT Query using LIKE Clause

```
SELECT Name, Class  
FROM students  
WHERE Name LIKE 'A%';
```

Explanation

This query selects the Name and Class of students whose names start with the letter “A”.

The % wildcard means:

- Zero, one, or many characters after “A”.

Examples of matching names:

- Aarav
- Aman
- Aayush

Important Points

- LIKE is used for pattern searching.
- % represents multiple characters.
- A% means names beginning with “A”.

b) UPDATE Query using LIKE Clause

```
UPDATE students  
SET Class = 12  
WHERE Name LIKE '_man';
```

Explanation

This query changes the Class to 12 for students whose names:

- Have exactly four letters

- End with “man”

The _ wildcard represents:

- One single unknown character.

Examples of matching names:

- Aman
- Iman

Important Points

- _ wildcard = exactly one character.
- _man means:
 - One character before “man”.

c) DELETE Query using LIKE Clause

```
DELETE FROM students  
WHERE Name LIKE '%esh';
```

Explanation

This query deletes records where the Name ends with “esh”.

The % wildcard means:

- Any number of characters before “esh”.

Examples of matching names:

- Rajesh
- Ramesh
- Suresh

Important Points

- %esh means names ending with “esh”.
- DELETE permanently removes records.

Report in MySQL

A report in MySQL is the organized presentation of data retrieved from one or more tables using SQL queries.

Reports help users:

- Understand data
- Analyze information
- Make decisions
- Generate summaries

MySQL itself mainly provides structured data. For advanced graphical reports, MySQL can be connected to tools like:

- Power BI
- Tableau
- Google Data Studio

Definition of Report

In simple terms, a report is the result of a query that extracts useful information from a database and presents it in a readable and meaningful format.

Purpose of Reports

Reports are used to:

- Analyze collected data
- Summarize information
- Support decision making
- Present information clearly

Types of Reports in MySQL

1. Summary Reports

Summary reports display summarized information such as:

- Total sales
- Total students
- Total marks

2. Detail Reports

Detail reports display complete detailed records.

Example:

- List of all students with marks

3. Grouped Reports

Grouped reports organize data into categories.

Example:

- Students grouped by class
- Employees grouped by department

4. Filtered Reports

Filtered reports display records based on conditions.

Example:

- Students scoring above 80%

5. Aggregate Reports

Aggregate reports use SQL functions such as:

- SUM()
- AVG()
- COUNT()

to summarize data.

Features of Reports in MySQL

1. Organized Data Presentation

Reports present information clearly and systematically.

2. Data Filtering

Reports can display only required data using conditions.

3. Summarization

Reports can summarize large amounts of data.

4. Multi-table Combination

Reports can combine data from multiple tables.

5. Timely Access

Reports provide quick access to information.

6. Easy Export

Reports can be exported into:

- PDF
- Excel
- CSV

formats.